

Отчет по лабораторной работе №7

Установка и конфигурация операционной системы на виртуальную
машину

Раднаве Ардан, НКАбд04-24
2026

Титульный лист

РУДН

Компьютерные и информационные науки, ФФМИЕН

Цель работы

Освоить на практике применение режима однократного гаммирования

Задание

Нужно подобрать ключ, чтобы получить сообщение «С Новым Годом, друзья!». Требуется разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Приложение должно: 1. Определить вид шифротекста при известном ключе и известном открытом тексте. 2. Определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста

Выполнение лабораторной работы

```
import random

def text_to_codes(text):
    """Преобразует строку в список кодов символов"""
    codes = []
    for i in range(len(text)):
        codes.append(ord(text[i]))
    return codes
```

Функция перевода тексты в числа

Требуется разработать программу, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Начнем с создания функции перевода текстовых символов в числовые значения

```
def xor(b1, b2):
    """XOR двух списков кодов"""
    if len(b1) != len(b2):
        print("Ошибка: длины не соответствуют")
        return None
    result = []
    for i in range(len(b1)):
        new = b1[i]^b2[i]
        result.append(new)
    return result
```

Функция для шифрования текста

```
def codes_to_text(codes):
    """Преобразует список кодов обратно в строку"""
    text = ""
    for i in range(len(codes)):
        text += chr(codes[i])
    return text
```

Подбор возможных ключей для фрагмента

Добавляю функцию генерации рандомного ключа

```
def generate_key(length):
    """Генерирует случайный ключ заданной длины"""
    key = []
    for i in range(length):
        key.append( random.randint(0, 255))
    return key
```

Генерация ключа

Проверка работы всех функций. Шифрование и дешифрование происходит верно, как и нахождение ключей, с помощью которых можно расшифровать верно только кусок текста


```

def text_to_codes(text):
    """Преобразует строку в список кодов символов"""
    codes = []
    for i in range(len(text)):
        codes.append(ord(text[i]))
    return codes

def codes_to_text(codes):
    """Преобразует список кодов обратно в строку"""
    text = ""
    for i in range(len(codes)):
        text += chr(codes[i])
    return text

def xor(b1, b2):
    """XOR двух списков кодов"""
    if len(b1) != len(b2):
        print("Ошибка: длины не соответствуют")
        return None
    result = []
    for i in range(len(b1)):
        new = b1[i]^b2[i]
        result.append(new)
    return result

def generate_key(length):
    """Генерирует случайный ключ заданной длины"""
    key = []
    for i in range(length):
        key.append( random.randint(0, 255))
    return key

# основная программа
plain = ("С Новым Годом, друзья!")
print(f"Открытый текст: {plain}")

# генерация ключа
key = generate_key(len(plain))
print(f"Ключ: {key}")

# преобразование ключа в коды
plain_codes = text_to_codes(plain)

# шифрование
cipher_bytes = xor(plain_codes, key)
cipher_text = codes_to_text(cipher_bytes)
print(f"Зашифрованный текст: {cipher_text}")

```

```
# дешифровка
decrypted_bytes = xor(cipher_bytes, key)
decrypted_text = codes_to_text(decrypted_bytes)
print(f"Расшифровано: {decrypted_text}")
```

```
# поиск ключа
target = ("С Новым Годом, друзья!")
print(f"Искомый текст: {target}")
target_bytes = text_to_codes(target)

possible_key = xor(cipher_bytes, target_bytes)
print(f"Найденный ключ: {possible_key}")
```

Выводы

В ходе выполнения данной лабораторной работы мной было освоено на практике применение режима однократного гаммирования.